SOFTWARE ENGINEERING MINI-PROJECT

AI-Powered Upskilling Recommendation System

Project 22 - Implementation and Review Package

PES University | Computer Science and Engineering (AI & ML)

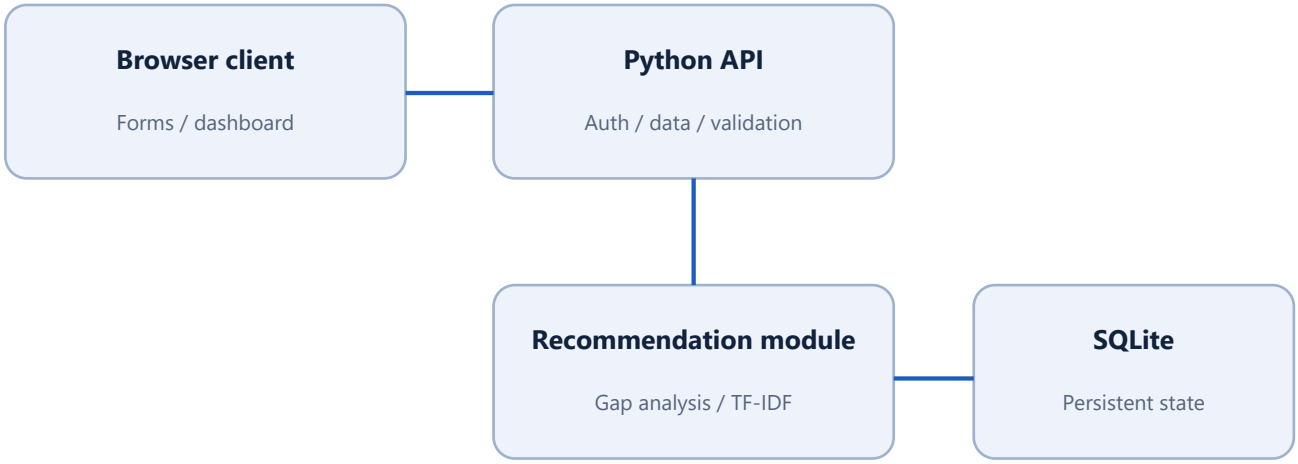
Team member	SRN
Poojitha V	PES2UG24AM113
Manasa C	PES2UG25AM803
Shanmuga	PES2UG24AM083
Manish S	PES2UG24AM085

Updated 26 October 2026. Includes a working application, newly created architecture and test plan, full SRS traceability, actual test evidence, sprint planning and demo recordings.

Independent teammate review, owner merge and formal sprint acceptance/freeze remain genuine team process actions. Sprint 1 material is recovery evidence; the final Sprint 2 working demo was captured on 26 October. Approval-gated merge and release are prepared.

Review map

Read the measured verification results first, then review the architecture, acceptance plan and all 48 requirement mappings. The user manual provides exact run and demonstration steps. Process decisions and limitations are included, with no backdated approvals.



Local coursework deployment: browser -> same-origin API -> SQLite

Source documents

SRS_Project_ID_22.pdf: 15 pages, original functional and non-functional requirements, use-case diagram and ER diagram. SE_Mini_Project_Delivereables_Part-2.pdf: backlog, sprint, PR/Actions, recording, documentation and freeze requirements. The new SAD/Test Plan document the actual code; the earlier submitted versions were unavailable.

Verification Evidence

Application baseline tested on 25 October; release gates and final Sprint 2 recording verified on 26 October 2026. These are actual observed results, with raw artifacts in evidence/. GitHub CI status must be checked for the published commit; a local pass does not assert remote CI success.

Check	Observed result	Artifact
Python API/security/regression	28 tests passed	python-tests.txt
JavaScript	Syntax check and DOM contract tests passed	CI / Python frontend contract
Real-browser student/admin workflows	Passed; no uncaught JS exceptions	browser-results.json
Mobile	390x844; no horizontal overflow	mobile-dashboard.png
Automated accessibility	0 WCAG 2.1 A/AA violations; 21 passing rules; 0 checks require review	accessibility.json
Recommendations	Synthetic 12-case MRR 1.0000, fixed-order 0.5097, hit@3 1.0000	recommender-evaluation.json
Serial dashboard timing	Median 5.0ms; p95 20.96ms; max 26.91ms	performance.json
Five-worker timing	Median 49.86ms; p95 160.17ms; max 282.34ms	performance.json
Outbound links	All 19 returned HTTP 200 during audit	link-audit.json
Backup/recovery	SQLite integrity ok; all table contents compared equal	backup-verification.json
Recordings	60.00s and 120.00s; full decode checks passed	videos/recording-metadata.json

The benchmark uses a small local catalog and disposable account; it is not production capacity certification. The synthetic ranking set is non-independent and does not measure real-student learning effectiveness. axe-core leaves human usability/screen-reader review outstanding. The current candidate is ready for independent review, not a fabricated team approval.

Release preparation verification (26 October)

Seven offline release-gate simulations pass (evidence/release-gate-tests.json). These are simulations, not an actual release. The new final Sprint 2 recording is 120.00 seconds and passes full decode verification; its actual capture timestamp is in videos/Sprint_2_Demo.metadata.json. Application code is unchanged from the previously passing baseline.

Software Architecture and Design (SAD)

Project 22 | Version 1.0 | Created 25 October 2026

Document status

This SAD was newly created with user authorization because the original submitted SAD was unavailable. It documents the implemented system and does not claim to reproduce an earlier submission. Sources are the 15-page repository SRS, its use-case and ER diagrams, and the Part-2 deliverables brief.

Problem, users and scope

Students maintain their academic profile, skills, qualifications and career goal. The product compares self-assessed skills with role targets, identifies missing and weak skills, generates a personal learning path and stores practice progress. Administrators manage student metadata, skills, career requirements and courses/resources. Opt-in milestone notifications are stored in the application.

Architecture

The same-origin HTML/CSS/JavaScript client calls a Python JSON API. SQLite persists application state. Recommendation computation runs on the server without an external AI service. Only explicitly allowed static assets are served.

Component	Responsibility	Main interface
index.html, style.css	Semantic forms, dashboard, responsive layout	Browser UI
app.js	Requests, safe text rendering and workflow state	Same-origin JSON
app.py	Routing, authentication, profiles, progress, transactions	API / make_server
administration.py	Validate and apply protected management operations	execute / ValidationError
recommendations.py	Curated data, TF-IDF ranking, projects, certificates	rank_learning
SQLite	Accounts, catalogs, sessions, progress and audit events	connect_db
tests, scripts	Regression, browser, accessibility and measured evidence	CLI commands

Deployment

One Python process listens on 127.0.0.1:8000; a browser connects over local HTTP. Python 3.10+ is sufficient for normal operation, with no mandatory third-party runtime dependency. HOST, PORT and UPSKILL_DB configure binding and storage. This is a local coursework demonstration server. Public hosting needs HTTPS, a production server/reverse proxy, abuse controls, monitoring and an independent deployment security review.

Database design

Table	Identity / principal information	Integrity
users	id; name; unique email; salt; password_hash; interests; goal; role; department; academic_year; phone	student role by default
skills	user_id + skill; proficiency level	student ownership; cascade deletion
skill_catalog	unique name	master skill list
career_roles	unique name	role identity
career_role_skills	role + skill; required_level	role cascade
portfolio	id; user_id; kind; title; skill	course/project/certification discriminator
learning_resources	id; skill; title; url; description; kind	admin-managed course/resource
recommendation_snapshots	user_id; JSON analysis; updated_at	latest persisted recommendation
progress	user_id + skill; completed_at	practice activity completion
sessions	token_hash; user_id; expires_at	raw token never stored
notification_preferences	user_id; enabled	disabled by default
notifications	id; user_id; title; created_at; read_at	private to student
audit_log	id; actor_id; action; target; created_at	admin mutation evidence

Foreign keys are enabled per connection. Writes are transactional and database handles close deterministically. Additive migrations preserve existing accounts and progress. In-use skills and selected career roles are protected from deletion. Student deletion cascades to their owned data and sessions.

Mapping to the SRS ER diagram

Student and Administrator share users with a role discriminator. Course, Project and Certification share portfolio with a kind discriminator. Unique skill/role names stand in for conceptual IDs. Proficiency_Level and Required_Level map to skills.level and career_role_skills.required_level. Recommendation uses a latest snapshot rather than an unbounded history. Progress represents one binary practice activity per skill; the dashboard computes its percentage for the chosen goal. The conceptual diagram includes optional provider, difficulty, issue-date and project-link metadata that is not separately modeled in this implementation. These decisions are documented deviations for the final demo.

Authentication and security

Passwords use random per-user salts and PBKDF2-HMAC-SHA256 with 300,000 iterations. Login compares derived digests in constant time and returns a random 256-bit bearer token for eight hours. Only SHA-256 token hashes are persisted. The browser keeps its token in sessionStorage; logout revokes it. Personalized queries always constrain user_id. Every admin mutation checks the database role before applying changes. Promotion is a local maintenance command, never a public endpoint.

JSON request bodies are limited to 16 KB. Values use parameterized SQL. Frontend user content uses textContent. Resource URLs require public HTTPS references and reject credentials, local/private numeric addresses and malformed ports; the server does not fetch submitted URLs. API replies use no-store and nosniff. Static replies use a Content Security Policy. These are tested controls, not a penetration-test certification. Password recovery, email verification, rate-limit infrastructure and public-deployment availability are outside this local baseline.

Gap and recommendation algorithm

Required skills are taken from the selected role. Missing means absent from the student's inventory; weak means present but below the role's required level. Candidate topics are the union of missing and weak skills. Readiness = 100 * requirements met / required skills. Practice progress = 100 * practiced required topics / required topics. No goal gives zero readiness and no career-specific path. Practice never automatically changes proficiency.

Each candidate document contains its skill and curated resource description. The query combines career goal, interests and portfolio titles. Tokenization uses lower-case alphanumeric terms. TF = 1 + ln(count); IDF = 1 + ln((1+n)/(1+document_frequency)). Cosine similarity ranks the vectors, with deterministic source order for ties. The latest custom resource for a skill overrides its default. A custom skill without a course receives an explicitly local practice project. Completed practice topics sort after unpracticed ones. Optional certifications are shown only for supported applicable roles with gaps, without asserting eligibility.

The method is content-based ranking using reference data, which the SRS permits. It is not a learned predictor of student outcomes. Twelve synthetic relevance cases compare ranking with fixed catalog order; the dataset, per-case results and limitations are preserved.

Main sequences

Student: register -> login -> save academic profile and goal -> save skills/levels -> inspect gaps/path -> add portfolio item -> log practice -> refresh/reprioritize. Administrator: local authorized promotion -> login -> change shared catalogs or student metadata -> inspect audit log. Confirmations precede destructive UI actions. All state is saved by the server before a success message.

API and error contract

Route group	Access	Purpose
GET /health, /api/catalog, /api/resources	Public	Liveness and shared reference data
POST /api/register, /api/login	Public	Create account / obtain session
GET /api/dashboard, /api/portfolio	Signed in	Own profile, analysis and qualifications
POST /api/profile, /api/skills	Signed in	Save own profile / skill inventory and levels
POST /api/portfolio/add, /api/portfolio/delete	Signed in	Manage own typed portfolio
POST /api/progress, /api/logout	Signed in	Practice state / session revocation
GET /api/notifications	Signed in	Preference and private events
POST /api/notifications/preferences, /api/notifications/read	Signed in	Opt-in and read own event
GET /api/admin/users, /api/admin/audit	Administrator	Student records and audit history
POST /api/admin/users, /api/admin/users-delete	Administrator	Edit metadata / delete student
POST /api/admin/skills, /api/admin/skills-delete	Administrator	Add / remove unused skill
POST /api/admin/roles, /api/admin/roles-delete	Administrator	Save target requirements / delete unused role
POST /api/admin/resources, /api/admin/resources-delete	Administrator	Add/edit/delete course or resource

Expected errors are JSON: 400 invalid data, 401 missing/expired credentials, 403 forbidden administration, 404 unknown route, 409 duplicate email. The UI displays errors in a live status region. Invalid input must not terminate the server.

Quality, operations and change control

A proposed local acceptance ceiling is two seconds for dashboard/recommendations; this makes the SRS's 'reasonable time' measurable without inventing an original SLA. The benchmark covers 30 serial requests and 30 with five workers. Responsive behavior is checked at 390px and desktop widths. axe-core checks detectable WCAG A/AA issues; human usability acceptance is separate.

Stop the server before copying the database or use scripts/backup_db.py for an online SQLite backup. Verify backup integrity, restore to a separate file and point UPSKILL_DB to it. Never publish databases, passwords or session tokens. CI runs Python regressions on Linux/Windows and a Chromium workflow. Independent teammate review and current checks are required before owner merge and formal release freeze.

Test Plan and Acceptance Criteria

New document created 25 October 2026 for the actual implementation. The earlier submitted test plan was unavailable; this plan does not claim to reproduce it.

Scope and environment

Verify FR-01 through FR-38 and NFR-01 through NFR-10. Use Python unittest for calculation/HTTP integration, Playwright for real browser workflows, axe-core for machine-detectable WCAG A/AA checks, and separate evaluation, benchmark and link-audit scripts. Local testing uses Windows, Python 3.12 and Chromium. CI covers Python 3.11/3.12 on Linux and Windows. Disposable example.com accounts and isolated SQLite databases keep test data separate from any real student information.

Entry and exit criteria

Entry: SRS available, application starts, schema initialized, expected requirement behavior identified. Exit: all automated regressions and browser acceptance pass; no known functional defect remains; documents and demo recordings match the tested product; independent teammate review and owner merge are recorded before final acceptance/freeze. A passing local suite is not a signed acceptance record.

Functional test cases

Case	Requirements	Action and expected result	Evidence
T01	FR-01,02	Valid registration succeeds; invalid name/email/password rejected; duplicate returns 409	test_app auth/validation; test_nfr
T02	FR-03,04	Login succeeds; wrong password denied; expired/revoked token denied	test_app auth/logout; test_nfr expiry
T03	FR-05,06,07	Save/edit academic profile, goal and contact information; reload retains it	test_completion academic profile; browser
T04	FR-08,09,10	Add/update/remove skills and levels; invalid levels rejected	test_completion weak skill; test_app
T05	FR-11,12,13	List roles; save valid goal; reject unknown role	catalog/profile tests; browser
T06	FR-14,15	Identify existing skills and role targets	gap_report; role lifecycle tests
T07	FR-16,17	Distinguish missing and below-target skill; correct readiness	test_completion academic_profile_and_weak_skill_gap
T08	FR-18,19	Rank gap topics deterministically using goal/context	test_sprint2 ranking; recommender evaluation
T09	FR-20	Course/resource metadata matches a missing/weak skill	metadata tests; link audit
T10	FR-21	Practical project prompt for every candidate, including custom skill	project tests; catalog lifecycle
T11	FR-22	Applicable certificate reference; omit unsupported path	test_sprint2 project_and_optional_certification
T12	FR-23,24,25	Resources categorized by skill and viewable via safe links	browser; frontend contract; link audit
T13	FR-26,27,28	Mark/unmark practice, compute percent and retain saved state	progress/persistence tests; browser
T14	FR-29	Practiced topics sort below unpracticed topics	test_app skills_goal_recommendations_and_progress
T15	FR-30,31,32,33	Goal, gaps, recommendations and progress visible	browser-results.json; screenshots
T16	FR-34	Admin edits student data; deleting student invalidates related data/sessions	test_completion admin_student_edit_and_cascading_account_delete
T17	FR-35	Admin adds/deletes unused skill; used skill protected	test_completion catalog_resource_and_role_lifecycle
T18	FR-36,38	Create/update role skills and target levels; protect selected role	completion lifecycle; test_sprint2 role edit

Case	Requirements	Action and expected result	Evidence
T19	FR-37	Add/edit/delete custom course/resource; analysis uses update	completion lifecycle; browser admin flow
T20	SRS diagrams	Portfolio private; opt-in milestone once; own read status	test_sprint2 portfolio/notifications

Non-functional criteria

ID	Proposed acceptance / method	Limits
NFR-01	All measured local dashboard requests under 2000ms; 30 serial, 30 with five workers	Project proposal, not an original numeric SRS SLA
NFR-02	Recommendation calculation included in dashboard timing	Small catalog/local hardware
NFR-03	Salted password hash and no stored raw session token	Control verification, not certification
NFR-04	Missing/expired credentials denied; data constrained to owner	API isolation cases
NFR-05	All eight admin mutation routes reject students/anonymous users	Explicit enumeration in test_completion
NFR-06	Zero axe-core WCAG 2.1 A/AA violations on tested student dashboard	Does not replace human usability review
NFR-07	Consistent anchors/forms; no horizontal overflow at 390px	Tested browser/viewport only
NFR-08	Deterministic available-data path; no-goal empty; custom-skill fallback	Curated reference approach
NFR-09	Saved data survives initialization and reload; backup integrity verified	Normal operation and local recovery smoke test
NFR-10	Separate modules; compile/API/browser regression repeatable	CI and documented design

Negative/security cases

Reject non-object JSON, invalid skill lists, boolean/out-of-range proficiency and oversized/invalid input. Deny missing/invalid/expired/revoked sessions and every student attempt at admin mutation. Verify another account cannot view/delete portfolio or read notifications. Reject javascript URLs, private numeric addresses, embedded URL credentials and malformed ports. Serve only allowlisted assets. Render user content as text.

Execution and evidence

The baseline Python suite contains 28 tests. Browser acceptance separately covers registration, login, academics, weak skills, portfolio, practice/reload, mobile overflow, admin skill/role/course changes and uncaught exceptions. JSON evidence records accessibility, recommendation reference cases, response times and outbound link status. Keep full Python output in evidence/python-tests.txt. Tests must run against the actual API and frontend, not mocks.

Run python -m unittest discover -s tests -v, node --check app.js, npm ci, npx playwright install chromium, and npm run test:browser. Run the three scripts evaluate_recommender.py, benchmark.py and audit_links.py to regenerate measured evidence. See USER_MANUAL.md for startup and admin setup.

Human acceptance and limitations

A real teammate must independently review the PR and verify the live demo. Human keyboard/screen-reader usability, real-student usefulness, cross-browser certification, penetration testing and public-deployment capacity are not claimed by automated checks. The synthetic relevance cases use catalog-like vocabulary and cannot establish real-world prediction accuracy. Scheduled sprint participation and owner approval cannot be generated by tests.

SRS Requirement Traceability Matrix

Source: SRS_Project_ID_22.pdf, pages 9-13 and diagrams pages 6/8. New SAD/Test Plan document the actual implementation. Implemented/tested describes the candidate, not independent acceptance.

ID	Original requirement	Implementation / evidence	Candidate status
FR-01	The system shall allow a new student to register.	app.py authentication; test_app auth; test_nfr	Implemented; review pending
FR-02	The system shall validate registration information.	app.py authentication; test_app auth; test_nfr	Implemented; review pending
FR-03	The system shall allow registered students to log in.	app.py authentication; test_app auth; test_nfr	Implemented; review pending
FR-04	The system shall display an error message for invalid login credentials.	app.py authentication; test_app auth; test_nfr	Implemented; review pending
FR-05	The system shall allow students to create a profile.	app.py profile; test_completion academic profile; browser	Implemented; review pending
FR-06	The system shall allow students to edit their profile.	app.py profile; test_completion academic profile; browser	Implemented; review pending
FR-07	The system shall store student academic and career information.	app.py profile; test_completion academic profile; browser	Implemented; review pending
FR-08	The system shall allow students to add skills.	app.py skills/levels; test_completion weak skill; browser	Implemented; review pending
FR-09	The system shall allow students to update or remove skills.	app.py skills/levels; test_completion weak skill; browser	Implemented; review pending
FR-10	The system shall store student skill information.	app.py skills/levels; test_completion weak skill; browser	Implemented; review pending
FR-11	The system shall display available career roles.	career_catalog/profile; test_app validation; browser	Implemented; review pending
FR-12	The student shall be able to select a career goal.	career_catalog/profile; test_app validation; browser	Implemented; review pending
FR-13	The system shall store the selected career goal.	career_catalog/profile; test_app validation; browser	Implemented; review pending
FR-14	The system shall identify the student's existing skills.	gap_report; test_completion academic profile; test_app gap_report	Implemented; review pending
FR-15	The system shall identify skills required for the selected career role.	gap_report; test_completion academic profile; test_app gap_report	Implemented; review pending
FR-16	The system shall identify missing or weak skills.	gap_report; test_completion academic profile; test_app gap_report	Implemented; review pending
FR-17	The system shall display the identified skill gaps.	gap_report; test_completion academic profile; test_app gap_report	Implemented; review pending
FR-18	The system shall analyze the student's skill gaps.	recommendations.py; test_sprint2 ranking/projects; evaluation JSON	Implemented; review pending

ID	Original requirement	Implementation / evidence	Candidate status
FR-19	The system shall generate personalized recommendations.	recommendations.py; test_sprint2 ranking/projects; evaluation JSON	Implemented; review pending
FR-20	The system shall recommend relevant courses and learning resources.	recommendations.py; test_sprint2 ranking/projects; evaluation JSON	Implemented; review pending
FR-21	The system shall recommend suitable projects.	recommendations.py; test_sprint2 ranking/projects; evaluation JSON	Implemented; review pending
FR-22	The system shall recommend relevant certifications where applicable.	recommendations.py; test_sprint2 ranking/projects; evaluation JSON	Implemented; review pending
FR-23	The system shall display recommended learning resources.	app.js cards; test_frontend_contract; browser; link audit	Implemented; review pending
FR-24	The system shall categorize learning resources according to skills.	app.js cards; test_frontend_contract; browser; link audit	Implemented; review pending
FR-25	The system shall allow students to view recommended resources.	app.js cards; test_frontend_contract; browser; link audit	Implemented; review pending
FR-26	The system shall allow students to mark learning activities as completed.	app.py progress/dashboard; test_app progress; test_completion persistence	Implemented; review pending
FR-27	The system shall calculate learning progress.	app.py progress/dashboard; test_app progress; test_completion persistence	Implemented; review pending
FR-28	The system shall display learning progress on the dashboard.	app.py progress/dashboard; test_app progress; test_completion persistence	Implemented; review pending
FR-29	The system shall update recommendations based on learning progress where applicable.	app.py progress/dashboard; test_app progress; test_completion persistence	Implemented; review pending
FR-30	The system shall display the student's career goal.	app.js render; browser-results and screenshots	Implemented; review pending
FR-31	The system shall display identified skill gaps.	app.js render; browser-results and screenshots	Implemented; review pending
FR-32	The system shall display recommended learning resources.	app.js render; browser-results and screenshots	Implemented; review pending
FR-33	The system shall display learning progress.	app.js render; browser-results and screenshots	Implemented; review pending
FR-34	The administrator shall manage student information.	administration.py; test_completion CRUD/authorization; browser	Implemented; review pending
FR-35	The administrator shall manage skills.	administration.py; test_completion CRUD/authorization; browser	Implemented; review pending
FR-36	The administrator shall manage career roles.	administration.py; test_completion CRUD/authorization; browser	Implemented; review pending
FR-37	The administrator shall manage courses and learning resources.	administration.py; test_completion CRUD/authorization; browser	Implemented; review pending
FR-38	The administrator shall update career-role skill requirements.	administration.py; test_completion CRUD/authorization; browser	Implemented; review pending

ID	Original requirement	Implementation / evidence	Candidate status
NFR-01	The system should respond to normal user requests within a reasonable time.	performance.json serial/concurrent benchmark	Verified in documented local scope
NFR-02	Recommendation results should be generated without unnecessary delay.	performance.json includes recommendations	Verified in documented local scope
NFR-03	User passwords shall be stored securely.	test_nfr salted password hash	Verified in documented local scope
NFR-04	Only authenticated users shall access their personal information.	test_sprint2 isolation; test_nfr expiry; test_app logout	Verified in documented local scope
NFR-05	Administrator functions shall be accessible only to authorized administrators.	test_completion all admin mutations	Verified in documented local scope
NFR-06	The system shall provide a simple and user-friendly interface.	browser acceptance; accessibility.json; human acceptance remains	Verified in documented local scope
NFR-07	Navigation between different pages shall be clear and consistent.	navigation anchors; mobile/desktop screenshots	Verified in documented local scope
NFR-08	The system should provide recommendations based on the available data.	deterministic rank tests; custom-skill fallback	Verified in documented local scope
NFR-09	Student progress should not be lost during normal system operation.	test_completion persistence; backup verification	Verified in documented local scope
NFR-10	The system should use modular components so that individual features can be updated easily.	separate modules, CI matrix, repeatable tests	Verified in documented local scope

Backlog and Sprint Plan

Original planning dates: backlog ETA 12 October, dry-run 12-16 October; Sprint 1 19-23 October; Sprint 2 26-30 October 2026. Recovery preparation occurred on 25 October, so no earlier completion date is asserted.

The linked GitHub Project is https://github.com/orgs/AI-UPSKILLING-TEAM/projects/1 . All 16 original issues are included with numeric Story points and a Sprint selection containing the scheduled dates. In Progress means implementation is prepared but independent review/process acceptance is open. This status does not claim a team-signed completion.

Issue assignment includes actual implementation author ManishS943 and the current review/demo owner. Organization membership/usernames were verified. Assignments do not fabricate contribution history: teammates should raise PRs only for work they actually perform.

Issue	Scope	Sprint	Points	Review/demo owner
#1	Registration/login	1	5	poojithav77
#2	Academic profile and goal	1	3	poojithav77
#3	Skill inventory and gap analysis	1	5	manasachandru12345
#4	Student dashboard and recommendations	1	5	poojithav77
#5	Automated tests / Actions	1	3	shanmugavel30072006-blip
#6	One-minute demo and PR review	1	3	poojithav77
#7	SRS/SAD/Test Plan alignment	1	3	ManishS943
#8	NFR baseline	1	3	shanmugavel30072006-blip
#9	Recommendation evaluation	2	8	manasachandru12345
#10	Curated courses/projects/certifications	2	5	manasachandru12345
#11	Persistent progress	2	3	manasachandru12345
#12	Notifications	2	3	shanmugavel30072006-blip
#13	Administrator management	2	5	ManishS943
#14	NFR/accessibility hardening	2	5	shanmugavel30072006-blip
#15	Integration and documentation	2	5	ManishS943
#16	Two-minute demo and freeze	2	3	ManishS943

Sprint 1 totals 30 points; Sprint 2 totals 37 points. These are planning estimates, not measured effort or completion percentages. PR #17 contains the candidate implementation. Independent teammate review, owner merge, in-period final Sprint-2 acceptance and a documented release freeze remain real team actions.

For future changes: contributor creates their own branch and PR, links the issue/requirement, waits for CI, obtains an independent owner/teammate review, addresses findings and merges only after approval. Keep actual timestamps and record deviations during the demo. The one-minute video is recovery evidence; a new final two-minute Sprint 2 demo was captured on 26 October. The older preview remains historical evidence.

Installation and User Manual

Project 22 | Python 3.10+ | Local browser application

Start

From the project folder run python app.py, or double-click START_WINDOWS.cmd. Open http://127.0.0.1:8000. No pip installation is required for normal use. The first run creates upskill.db and seeds six career roles and sixteen skills. Press Ctrl+C to stop. HOST, PORT and UPSKILL_DB optionally change binding and database location.

Student workflow

1. Register with a name, valid email and an 8-128 character password, then sign in. Duplicate emails and invalid credentials are rejected. Use demonstration data when sharing the screen.

2. Save your name, career goal, department/course, academic year, optional phone and interests. Select a role before expecting career-specific recommendations.

3. Check existing skills and choose Beginner (1), Intermediate (2) or Advanced (3), then save. Unchecking removes a skill. Checked skills can still be weak when below the role's target. Readiness measures self-reported requirements met, not employment eligibility.

4. Add completed courses, projects and certifications to the portfolio with title and related skill. Remove entries when needed. Portfolio titles inform recommendation context but do not automatically prove skill proficiency.

5. Inspect missing/weak gaps. Follow recommended course/resource links and practical project prompts. Optional certifications appear only where applicable; consult provider requirements directly.

6. Mark required topics Practiced as you complete them. Unchecking reverses completion. Progress is role-relative and survives refresh/restart. Switching goal changes the denominator while preserving stored topics. Practice completion does not change proficiency.

7. Enable in-app notifications to receive a new milestone event once per newly completed topic. Mark events read. No email/SMS service is used. Sign out to revoke the session.

Administrator setup

Register the account normally, then run python app.py promote-admin your-account@example.com against the same database. Refresh or sign in again. Only a person with local server/database authority should run that command. No default admin password or remote promotion endpoint exists.

Administrator workflow

Create skills in Skill catalog; in-use skills are protected from deletion. Create/update role requirements using comma-separated names with optional target levels, such as Python:2, SQL:3. Edit loads existing values; selected roles cannot be deleted. Create courses/resources using a catalog skill, title, public HTTPS URL and description. Edit loads the same id for updating. Delete removes the custom row; built-in reference data remains a fallback. Student forms support name, department and year edits; deleting an account removes its private data and sessions. Shared-data changes appear in the audit log.

Troubleshooting and recovery

If the port is in use, select an available PORT and use that address. If a session expires, sign in again. Resolve user/role/resource/portfolio/progress references before removing a skill; reassign affected goals before removing a role. A new skill without a course has a practice project until an administrator adds a learning link. Refresh restores saved data, not unsaved edits. Back up with python scripts/backup_db.py backup.db; restore to a separate file and select it with UPSKILL_DB.

Recommendation Method and Evaluation Limits

The SRS permits appropriate training or reference data. This system uses curated reference data and TF-IDF cosine similarity: a content-based recommendation method. No trained predictor, external LLM or student-outcome model is claimed.

Candidates are missing/weak skills for the chosen career goal. Documents combine skill and course descriptions; query context combines goal, interests and portfolio titles. Natural-log term frequency and smoothed IDF produce vectors; cosine similarity ranks them, with deterministic source-order ties. Practiced topics sort later without implying mastery. A custom skill without a course still receives an original practice-project prompt; an administrator can add a course URL.

The seeded catalog has 16 skills and six roles. Three supported role-specific credential references are optional; pricing and eligibility are not assessed. All 19 reference links are checked in evidence/link-audit.json. Project prompts are small original exercises, not accredited assignments.

Evaluation uses 12 author-defined synthetic cases, two per role, one relevant skill per case. The full reference set, expected topic and observed ranking are retained in evidence/recommender-evaluation.json. Mean reciprocal rank is the mean inverse rank of the first relevant result. Hit@3 is the share of cases with that result in the first three. Fixed catalog order is the baseline. Current MRR is 1.0000 versus 0.5097 for fixed order; hit@3 is 1.0000.

These small, non-independent, catalog-derived cases are a reproducibility check, not 100% real-world accuracy. Independent evaluation needs diverse student descriptions, teacher/student relevance judgments, a frozen held-out dataset and learning-outcome follow-up. Readiness is a self-reported checklist, not an academic or employment eligibility decision.

Demonstration Recordings and Speaking Guide

Recordings are genuine automated browser captures of the running local product using disposable accounts. Sprint 1 recovery evidence and the historical Sprint 2 preview were captured on 25 October. A new final Sprint 2 working-product demonstration was captured on 26 October, within the scheduled 26-30 October week. Actual team review/acceptance is recorded separately.

One-minute Sprint 1 demo

evidence/videos/Sprint_1_Demo.mp4: 0-12s app/register/login; 12-28s academic profile, career goal and skills; 28-39s gaps and learning cards; 39-51s practice and notification; 51-60s reload/persistence. Explain what is saved, why topics are recommended and why practice differs from proficiency.

Two-minute Sprint 2 working-product demo

evidence/videos/Sprint_2_Demo.mp4: 0-28s student login, Cloud Engineer goal and weak proficiency; 28-49s gaps, projects and portfolio; 49-70s progress, notifications and reload; 70-102s administrator course/role management; 102-111s audit events; 111-120s updated path. Captions identify the workflow and actual capture date. The earlier Sprint_2_Preview.mp4 remains historical preparation evidence. No real credentials appear.

Final live demonstration

Start from a fresh folder, register a student, save academics/goal/levels, show missing and weak skills, add a portfolio item, complete/undo practice and refresh. Demonstrate admin authorization and management. Present the new SAD/Test Plan, traceability, actual test evidence, synthetic-ranking limitations, project board and PR checks. Explain deviations. Review the final Sprint-2 recording with the team and record genuine acceptance.

Design Decisions, Deviations and Process Status

Implementation prepared 25 October; final Sprint 2 demo and release automation prepared 26 October 2026. No work, commit, recording or approval is backdated.

Implemented

Authentication, academic profile, skill/proficiency management, goal selection, missing/weak gap analysis, content-based learning recommendations, courses/resources, practical project prompts, optional certification references, private portfolio, persistent practice progress, dashboard, notifications and administrator management are implemented and tested. See TEST_EVIDENCE.md.

Explicit decisions

1. TF-IDF uses curated reference data, as permitted by the SRS constraint. It is not a learned predictor.

2. Readiness compares self-reported skill levels with role targets. Progress is binary skill-based practice completion; neither proves mastery.

3. Course/project/certification records share a typed portfolio table. Student/admin accounts share role-based users. Recommendations retain a latest JSON snapshot.

4. Some optional metadata in the conceptual SRS ER diagram (provider, difficulty, issue date and project link) is not separately modeled. Normative FR behavior uses title, type, related skill and resource descriptions.

5. Admin-created courses/resources can be edited/deleted; version-controlled reference entries remain fallbacks. Latest custom resource per skill overrides its default.

6. Notifications are in-app. The SRS does not require email/SMS/push delivery.

7. SAD and Test Plan were newly created with user authorization because the earlier originals were unavailable. Equivalence to prior submissions cannot be verified.

Genuine team/process actions

Sprint 1 was scheduled 19-23 October; current recovery work is dated 25 October. Sprint 2 is scheduled 26-30 October. Sprint_2_Demo.mp4 was genuinely captured on 26 October and now supplies the in-period working-product video. The older 25 October preview is retained as historical evidence. Another authorized teammate must review the actual PR; an owner must merge after current checks pass. Teammates must raise their own contribution PRs for work they actually do. Current assignments allocate review/demo responsibility and do not manufacture contribution history. Formal freeze requires real acceptance and an owner-recorded final commit/date.

The application and review package are prepared; independent review, merge, in-period Sprint-2 acceptance and formal freeze remain visible process actions. See RELEASE_AND_FREEZE_CHECKLIST.md.

Review and release automation

All three other team members have review requests. Main requires one independent approval and six CI status contexts; stale approvals are dismissed and administrator bypass is disabled. Auto-merge is enabled at repository level and is queued on PR #17 after publication. The accepted-release workflow waits for the actual reviewed merge and passing main CI, verifies the approval on the final PR revision, then creates v1.0.0, verifies and attaches report/videos, publishes the immutable release and closes issues #1-#16. Seven offline gate simulations pass; a real release has not been executed before team approval.

Release Candidate and Development Freeze

Current preparation: 26 October 2026. The application, documents, 60-second Sprint 1 recovery video and new in-period 120-second Sprint 2 working-product demo are ready for actual review.

Enforced acceptance gate

Main requires a pull request, one genuine independent approval, four Python matrix checks, browser-tests and outline. Stale approvals are dismissed; administrators cannot bypass this rule. The source author is ManishS943. Poojitha, Manasa and Shanmuga have review requests. Approval of PR #17 should cover code, the new SAD/Test Plan and the explicitly documented design decisions.

Queued completion

GitHub auto-merge waits for a real approval and all required checks. After the merge, the Project verification workflow runs against main. Only a successful main push for PR #17's actual merge commit can trigger Accepted release and freeze. That workflow independently checks that the final PR revision has a non-author approval, creates an annotated v1.0.0 tag without moving an existing tag, uploads and SHA-256-verifies the report and both final demo videos, and publishes the release. Published release tags/assets are configured immutable. It then closes the 16 accepted backlog issues. The GitHub Project's closed-issue workflow updates their status.

Final evidence location

The actual reviewer, accepted merge SHA, passing main run and release/freeze timestamp will be recorded at https://github.com/AI-UPSKILLING-TEAM/AI-Upskilling-Recommendation-System/releases/tag/v1.0.0 after the gate is satisfied. Until that release exists, independent approval, merge and final publication remain pending; this file does not invent them.

Verification and limits

Seven offline release-gate simulations pass, covering missing, author-only and stale approval, foreign repository, unrelated merge, conflicting tag and the accepted publication order. These verify logic without contacting GitHub and do not claim that a real release was published. Future fixes require a separate reviewed version; the frozen tag is never moved.

Reference-link audit

All 19 reference links returned HTTP 200 at the recorded audit. Availability may change. This is a reachability check, not credential eligibility or course accreditation review.

Reference	URL	Status
Python official tutorial	https://docs.python.org/3/tutorial/	200
Microsoft Learn: Introduction to Transact-SQL	https://learn.microsoft.com/en-us/training/modules/introduction-to-transact-sql/	200
Microsoft Excel training	https://support.microsoft.com/en-us/excel	200
Khan Academy: Statistics and probability	https://www.khanacademy.org/math/statistics-probability	200
Matplotlib tutorials	https://matplotlib.org/stable/tutorials/index.html	200
Google Machine Learning Crash Course	https://developers.google.com/machine-learning/crash-course	200
MDN: Learn web development	https://developer.mozilla.org/en-US/docs/Learn_web_development	200
MDN: CSS styling basics	https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics	200
MDN: JavaScript	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide	200
Pro Git book	https://git-scm.com/book/en/v2	200
MDN: Web APIs	https://developer.mozilla.org/en-US/docs/Web/API	200
Python unittest documentation	https://docs.python.org/3/library/unittest.html	200
Ubuntu command-line introduction	https://ubuntu.com/tutorials/command-line-for-beginners	200
Cloudflare: Learning center	https://www.cloudflare.com/learning/	200
Microsoft Learn: Azure fundamentals	https://learn.microsoft.com/en-us/training/paths/microsoft-azure-fundamentals-describe-cloud-concepts/	200
OWASP Top 10	https://owasp.org/www-project-top-ten/	200
Microsoft Certified: Power BI Data Analyst Associate	https://learn.microsoft.com/en-us/credentials/certifications/data-analyst-associate/	200
AWS Certified Cloud Practitioner	https://aws.amazon.com/certification/certified-cloud-practitioner/	200
ISC2 Certified in Cybersecurity (CC)	https://www.isc2.org/Certifications	200